* Steps to Create Free AWS Educate Account

step 1: Visit AWS Portal
- Go to AWS Educate
- Click on "Join AWS Educate" at the top right corner

step 2: Select your Role
- Choose student
- Click next to proceed

step 3: Fill the Personal Information
- Enter your name, email and country
- Select whether you are affiliated with an educational institution
- If yes, enter your university name
- Set a password and accept the terms and conditions

step 4: Varify your Email
- AWS will send a verification link to your email.
- Open the email and click on verificati -on link.

step 5: log in to AWS Educate
- Once verified, go back to AWS Educate and log in.
- You will now have access to AWS Educate resources and free credits.

Introduction to Cloud

- Cloud computing is the delivery of computing server over the internet. Or "the cloud".

Summary:

* Understanding cloud computing and its benefits
- Benefits: Scalability, flexibility, cost-effectiveness, security and reliability.

* Cloud Service and Deployment model:
IaaS: provides virtual machine, storage and networking eg: AWS EC2, S3
PaaS: Provides a platform for application development eg: Gmail, Dropbox
SaaS: Provides software application over the internet. eg: AWS, google cloud.

Public Cloud: services shared across multiple customers eg: AWS, google cloud
Private Cloud: cloud infrastructure dedicated to a single organization.
Hybrid Cloud: A combination of public and private cloud for flexibility.

* AWS Global Infrastructure:
- AWS regions: Geographical areas where AWS hosts its data centers.
- Availability zones: Multiple data centers
- Edge locations: AWS content delivery network for faster data access.

* Shared Responsibility Model:
- AWS Responsibilities: Securing infrastructure
- User Responsibilities: Managing data, identity applications and security configuration.

* AWS Core Services Overview:
- Compute Services : Amazon EC2, AWS Lambda
- Storage Services : Amazon S3, Amazon EBS
- Database Services : Amazon RDS, Amazon Dynamo DB
- Networking Services : Amazon VPC, AWS CloudFront

* Hands-on Practic in a lab Environment:
- Setting up an AWS Educate
- Launching and configuration an EC2 instance
- Storing and retrieving files using Amazon S3
- Creating a simple database in RDS
- Running a basic AWS lambda function.

Google Cloud

1. Creating a Free Account:

Visit https://cloud.google.com/free

- Click on Get started for free
- Sign in with Gmail Id
  - Select Country as India → Agree and Continue
- Click on create new payment profile and fill the details
  → Profile type, Name, address, pincode etc.
- After filling the details. Click on Payment method:
- Click on pay by UPI or QR code and Click on start free
- Scan the QR Code through any of GPay Pay-TM etc
- Account gets will be created and will be landing at google cloud Console
- Click on Try it on console
- Account is created and ready to use free account for 90 days

Experiment-1: Creating a Virtual Machine:
Configure and deploy a virtual machine with specific CPU and memory requirements in Google Cloud.

=>
- Click in search button/bar and type Compute Engine or select Compute Engine
- Click on create Instance
- Give the name for VM instance -> specified as vm1 (can be any name typed in lower case)
- Select the region -> US Central (low) is selected, any region can be selected and any zone can be selected from that region.

- Configure the Virtual Machine based on vCPUs and Memory needed -> E2 series is selected currently which is low cost and used for day to day computing.
- Any Virtual Machine configuration can be selected based on requirement.

- Click on machine Type -> Select the required machine type shared-core e2-micro is selected since it costs less.
- Keep advanced configuration default:
- Check the cost estimate on right side panel which will indicate the monthly cost for running the VM of defined configuration.

- Click on create and wait for the vm to be created.
- Connecting to VM:
- Click on vm1

- Click on SSH and select open in browser window
- Click on Authorize
- Virtual Machine named VM1 with linux OS is available for access.
  - Task - Execute some basic tasks
- Close the window and back to VM instance
- Click on more option specified with 3 dots on delete click (to delete the instance)
- Click on the notification for complete deletion.

2. Getting started with Cloud Shell and gcloud:
Discover the use of gcloud commands to mana-ge google Cloud resources from cloud Shell.

⇒ Step 1: Open cloud shell
- Click the sign into Google cloud Console: https://console.cloud.google.com/
- Click the cloud Shell icon in top-right corner
- A terminal will open at bottom of page

Step 2: Initialize gcloud CLI
- Run the foln command in shell: gcloud init
- Follow prompts to:
  - Authenticate your Google Account
  - Select a google cloud project

Step 3: Varify gcloud Setup
- To check if gcloud is properly configured, run: gcloud config list
- This displays current project, account & region setting.

Step 4: List Available Project
- Run the command to view all projects
  gcloud projects list

Step 5: Set Active Project: To set specific project as the active one, run: gcloud config set project PROJECT_ID
Replace PROJECT_ID with your actual project ID.

Step 6: Check Authentication Status
- Run command to varify that you're authenticated: gcloud auth list

Step 7: Create a VM Instance

Step 8: List Running VM Instance
To Check all running instances, run: gcloud compute instances list

Step 9: Deleta a VM Instance

Step 10: Enable an API: run:
gcloud services enable compute.googleapis.com

Step11: Deploy an Application to App Engine

If you have an appl^{n} ready, deploy it using:

gcloud app deploy

follow the instruction to deploy & access app.

Step12: View Active Billing Accounts

run: gcloud beta billing accounts list

Step13: Exit Cloud Shell

3. Cloud functions: Create and deploy a Cloud function to automate a specific task based on Cloud Storage event.

Step 1: Enable Required APIs
Before deploying the Cloud function, enable necessary APIs:
gcloud services enable cloudfunctions.googleapis.com storage.googleapis.com

Step 2: create a Cloud Storage Bucket
Create one:
gcloud storage buckets create BUCKET_NAME --location=US-central1
Replace BUCKATE_NAME with a unique name for your bucket.

Step 3: Write the Cloud function code
1. Open Cloud Shell and create a working directory
mkdir gcs-function && cd gcs-function
2. Create and open a new Python file:
nano main.py
3. Add the foln code inside main.py:
import functions_framework
@functions_framework.cloud_event
def gcs_trigger(cloud_event):
data = cloud_even.data
bucket = data["bucket"]
file_name = data["name"]
print(f" file {file_name} upload to {bucket}")
4. Save & close the file (CTRL+X, Y, Enter)

Step 4: Create a requirement.txt file
Create and open a requirements.txt file
nano requirements.txt

Add requirement dependency:
functions-framework
Save & Close (CTRL + X, Y, Enter)

Steps: Deploy the Cloud function

Run the following command to deploy the function:

```
gcloud functions deploy gcs-trigger \
  --gen2 \
  --runtime=python311 \
  --region=us-central1 \
  --source=. \
  --entry-point=gcs_trigger \
  --trigger-event-filters="type=google.cloud.storage.object.v1.finalized" \
  --trigger-even-filters="bucket=BUCKET_NAME" \
  --allow-unauthenticated
```

Replace BUCKET_NAME with your actual Cloud Storage bucket name.

Steps: Test the Cloud function

1. Upload the file to Cloud Storage bucket:

```
gcloud storage cp test-file.txt gs://BUCKET_NAME
```

2. Checks logs to verify function execution:

```
gcloud functions logs read gcs_trigger --region=us-central1
```

Program - 4

App Engine: Deploy a web application on App Engine with automatic scaling enabled.

=) Step 1: Enable Required APIs

Before deploying your application, enable the App Engine API:

gcloud services enable appengine.googleapis.com

Step 2: Create an App Engine Application

Create App Engine

gcloud app create --region=us-central1

Step 3: Create a simple web application

1. Open cloud shell & create a project directory

mkdir app-engine-demo && cd app-engine-demo

2. Create a python file: nano main.py

3. Add the following simple flask application code

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
    return "Welcome to Google App Engine with Auto Scaling!"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

4. Save & close

Step 4: Create a requirements.txt file:

nano requirements.txt

Add foll^n dependicies

```
Flask
gunicorn
```

Save & Close

Step 5: Create an app.yaml File for App Engine

nano app.yml

Add the following content:

```
runtime: python311
entrypoint: gunicorn -b :$PORT main:app
automatic_scaling:
  min_instances: 1
  max_instances: 5
  target_cpu_utilization: 0.65
  target_throughput_utilization: 0.75
```

Save & close

Step 6: Deploy the web application

Run the following command to deploy

gcloud app deploy

- Confirm the deployment when prompted (y).

Step 7: Access the Deployed Application

Open deployed, open your web app in a browser.

gcloud app browse

This will return a URL, where you can view your running app.

Step 8: View Logs & Monitor Scaling

Check the logs of your application:

gcloud app logs tail -s default

Monitor the deployed services:

gcloud app services list

5. Cloud storage : Quikstart : Google Cloud Stor -age provides scalable and secure object storage for managing data, accessible via the cloud console or gustil CLI.

Step 1: Open Google Cloud Console

Create a new project name progos.

Step 2: Enable Cloud Storage API (If not enabled)

(a) In the google cloud console, click the navigat -ion Menu on the top left.

(b) Go to APIs & Services → Library

(c) Search for cloud storage api.

(d) Click Enable if it is not already enabled.

3. Create a cloud storage Bucket

1. In navigation bar Menu, go to storage → Buckets
2. Click Create Bucket
3. Pick a name, say progSspk
4. Chose where to store your data
   (i) Select radio button region: US-central1
5. Choose how to store your data
   (ii) Set a default class
   Standard
6. Choose how to control access to objects

Undo: "Enforce public access prevention on this bucket"

Access control: Choose uniform

7. Click Create

Step 4: Click upload and click a file to upload

One can upload one more file: By clicking displa -yed on top of the region which displays "1 file successfully uploaded".

Step5: Now download one of the files:
Click on the square box next to file name one wants to choose in the displayed list of files.
Then click download just on top of this list of files.

Step6: Make a file public

(a) open your bucket & click on the file.
(b) Click the permission tab
(c) Click add principle
(d) In the new Principals field, enter: allUsers.
(e) Select assign roles:
type roles/storage.objectViewer
(f) ~~Ado~~ Click save.

Now your file publicly accessible.
You will see a public URL
Anyone can access the file using this link

Step7: Delecte a File or Bucket

- To delete a file, click on the file and Select Delete.
- To delete a bucket, go to Storage → Buckets, select the bucket and click Delete

6. Cloud SQL for MySQL: Discover how Google Cloud SQL for MySQL provide automated management and high availability for MySQL databases?

Step 1: ENable Cloud SQL API

1. Open Google Cloud Console.
2. Go to APIs & Services → Library
3. Search for Cloud SQL Admin API and click Enable

Step 2: Create a Cloud SQL for MySQL Instance

1. Go to Navigation Menu → SQL
2. Click create Instance → Choose MySQL.
3. Set:
   - Instance ID
   - Password
   - Region and Zone
   - Machine Type
   - Storage Capacity
4. Click create and wait for the instance to intialize.

Step 3: Connect to Cloud SQL

⇒ Using Cloud Console

- Open SQL → Click on your instance
- Under Connections, find Public IP or Private IP
- Use the Cloud SQL Auth Proxy or MySQL client to connect.

⇒ Using MySQL Client

gcloud sql connect my-~~sql~~mysql-instance --user=root

or

mysql -u root -p -h [INSTANCE_IP]

Replace [INSTANCE_IP] with the actual instance IP

• Using Django / Flask

```
DATABASES = {
 'default': {
  'ENGINE': 'django.db.backends.mysql',
  'NAME': 'your-db-name',
  'USER': 'root',
  'PASSWORD': 'your-password'
  'HOST': '/cloudsql/your-proj-id:your-region:your-instance',
  'PORT': '3306',
  }
}
```

Step 4: Enable High Availability (Optional)

Step 5: Create a Read Replica (Optional)

Step 6 :

- Enable Automated Backups
  1. Open your instance → Click Backups.
  2. Click Edit → Enable automatic
- Manually Create a Backup

```
gcloud sql backups create --instance = my-mysql-instance
```

- Restore from Backup

```
gcloud sql backups restore BACKUP_ID
    --instance = my-mysql-instance
```

7. Cloud Pub/Sub: Experiment how Google Cloud Pub/Sub facilitate real-time messaging and communication between distributed applications.

Step 1: Enable Cloud Pub/Sub API

1. Open Google Cloud console → Navigation Menu → APIs & Services → Library
2. Search for Cloud Pub/Sub API and click Enable

Step 2: Create a Pub/Sub Topic

- Go to Navigation Menu → Pub/Sub → Topics
- Click Create Topic
- Enter a Topic ID
- Click Create.

Your Topic is now ready.

Step 3: Create a Subscription

1. Click on your Topic → Create Subscription
2. Enter a Subscription ID
3. Choose a Delivery Type:
   - Pull - Messages are manually fetched by the Subscriber.
   - Push - Msgs are automatically sent to an HTTP endpoint
4. Click Create

Step 4: Public a Message (Using gcloud CLI)

Run the following in Cloud Shell:

gcloud pubsub topics publish my-topic --message "Hello. Pub/Sub!"

- Message published Successfully

Step 5: Pull Messages from Subscripti-on (Using gcloud CLI)

Run the following command:

gcloud pubsub subscriptions pull my-subscription --auto-ack

This will retrieve and acknowledge msg from my-subscription

• You will see the msg : Hello Pub|Sub!

Step 6: Publish & Subscribe Using Python (optional)

Output:

8. Multiple VPC Networks: Explore benefits of using multiple VPC networks in Google Cloud for organizing and isolating resources.

Step 1: Create a VPC Network

- Go to google cloud console → Navigation Menu → VPC Network → Create VPC Network
- Specify the name, region and subnet configuration for your VPC
- Click Create

Step 2: Create Additional VPC Networks

- You can repeat the process to create as many VPCs as needed.
- Choose Custom Subnet mode to define your own subnets or Auto mode for auto-assigned subnets.

Step 3: Set Up VPC Peering (Optional)

- Go to VPC Network Peering → Create Peering Connection.
- Select the Source VPC & Destination VPC
- Define the network and routes that can be shared acoross the VPCs.
- Click Create.

Step 4: Create Firewall Rules (Optional)

Step 5: Set Up Shared VPC (optional)

9. Cloud Monitoring: Discover how cloud Monitoring help in tracking and analyzing the performance and health of Cloud resources?

Step 1: Enable Cloud Monitoring API

- Open the Google Cloud Console → Navigation Menu → APIs & Services → Library
- Search for Cloud Monitoring API and Click Enable.

Step 2: Set Up Monitoring for Your Resources

- Go to Navigation Menu → Monitoring → Dashboards.
- Click on Create Dashboard to start building your custom dashboard.
- Select Metrics from the dropdown & Choose the service you want to monitor.
- Add the required metrics to your dashboard and adjust visualizations like line charts, heat maps or bar charts.

Step 3: Set up Alerts

1. Go go Navigation Menu → Monitoring → Alerting
2. Click create Policy
3. Choose the condition (CPU usage > 80%)
4. Select the notification channels where the alert should be sent.
5. Set up escalation policies to ensure the right team is notified.
6. Click Create to finish the alert policy.

Step 4: Review Logs

- Go to navigation Menu → Logging.
- Choose the resource type and define your log filter
- Use Log Explorer to search for specific logs, such as error messages or performance warnings, and correlate them with metrics in Cloud Monitoring

Step 5: Set Up Distributed Tracking (optional)

Output:

10. Steps to deploy a Containerized Application to GKE

* Set Up Google Cloud SDK & Kubernetes Tools
  1. Install Google cloud SDK
  ~~2.~~ Google Cloud SDK Installation
  2. Install kubectl
  (if you have SDK installed, you already have kubectl)

* Create a Google Cloud project
  1. Create a new Google Cloud project (if you don't have)
  2. Set your project in gCloud CLI:
  gcloud config set project PROJECT_ID

* Enable Required APIs
  1. Enable kubernetes Engine API:
  gcloud services enable container.googleapis.com
  2. Enable Compute Engine API
  gcloud services enable compute.googleapis.com

* Create a Containerized Application
  1. Create a Dockerfile for your application.
  Dockerfile: (Eg)
  dockerfile
  CopyEdit
  FROM node:14
  WORKDIR /usr/src/app
  COPY...
  RUN npm install
  EXPOSE 8080
  CMD ["npm", "start"]

2. Build a Docker image :

```
docker build -t gcr.io/PROJECT_ID/my-app:v1
```

(Replace your proj ID)

3. Push the image to google Container Registry

```
docker push gcr.io/PROJECT_ID/my-app:v1
```

⇒Create a Kubernetes Deployment

Step 1: Create a Kubernetes Deployment Configuration file

deployment.yml (for containerized application)

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: my-app
spec:
  replicas: 3
  selector:
     matchLabels:
      app: my-app
template:
 metadata:
  labels:
    app: my-app
spec:
 containers:
 - name: my-app
  image: gcr.io/PROJECT_ID/my-app:v1
  ports:
   - containerPort: 8080
```

Replace PROJECT_ID with your project.ID.

Step2: Apply Deployment to the kubernetes cluster

kubectl apply -f deployment.yaml

~~kuberne~~

Expose the Application via a Service

Step1: Create a Service to expose the application

Eg: Service.yaml for external exposure

```
apiVersion: V1
kind: Service
metadata:
  name: my-app-service
  spec:
    selector:
      app: my-app
ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
type: LoadBalancer
```

Step 2: Apply the Service Configuration:

```
kubectl apply -f service.yaml
```

Step 3: Get the external IP address

```
kubectl get svc
```

* Varify the Application

1. Open a web browser and navigate to external IP address to varify your application is running
2. You can also use kubectl to get the status of your pods & services

```
kubectl get pods
kubectl get svc
```